TermType: A Zero-Dependency Terminal-Based Real-Time Typing Tutor

1st Mann Vaswani
Roll No. 230132
Rishihood University
mann.v23csai@nst.rishihood.edu.in

2nd Shiven Ahuja
Roll No. 230114
Rishihood University
shiven.a23csai@nst.rishihood.edu.in

***Abstract*—Typing proficiency remains a critical skill in the modern computing landscape. While existing typing applications are feature-rich, they often rely heavily on modern high-level libraries, graphical user interfaces, or resource-heavy web engines. This paper introduces TermType, a minimalist, terminal-based typing tutor built entirely in C from scratch. TermType is distinguished by its strict adherence to low-level systems programming constraints: it operates entirely without standard C utilities such as `<string.h>`, `<math.h>`, or system-level dynamic memory allocation like `malloc()`. Instead, it leverages custom implementations for bump-allocated memory pooling, string manipulation, and integer arithmetic. Operating within a POSIX raw terminal environment, TermType provides responsive real-time analytics including adjusted Words Per Minute (WPM), keystroke-level accuracy, and granular per-character performance tracking. A frame-buffered rendering pipeline eliminates terminal flicker by batching all output into a single `write()` syscall per frame. This paper details the methodology behind designing a zero-dependency architecture and discusses the efficacy of the application in delivering instantaneous typing feedback without the computational overhead of standard runtime environments.**

***Index Terms*—Systems Programming, C Language, Terminal Applications, Memory Management, Typing Tutor, POSIX**

I. Introduction

The development of typing tutors has predominantly shifted toward web-based platforms and heavy desktop applications. While accessible, these applications obscure the fundamental mechanisms of I/O handling, real-time user event processing, and memory management. TermType was conceived as an educational systems-programming endeavor to rebuild a real-time application from first principles. By enforcing a constraint against standard library usage, TermType proves that modern, responsive experiences can be engineered using bare-metal concepts. The application additionally exposes fine-grained statistics aligned with the MonkeyType scoring model, distinguishing between adjusted WPM (counting only fully correct words) and raw keystroke accuracy, thereby providing more meaningful feedback to the user.

II. Methodology

The development of TermType was divided into distinct architectural components, each designed to replace standard system functionality with custom implementations.

A. Zero-Standard-Library Constraint

To achieve a minimal footprint and maximum control over execution, TermType eschews standard C libraries. Functions typically provided by `<string.h>` (e.g., `strlen`, `strcmp`) and `<math.h>` were hand-rolled inside `string.c` and `math.c` respectively. This constraint forces an intimate understanding of data representation and pointer arithmetic. The only external system headers permitted are `<unistd.h>` for raw `write()` and `read()` syscalls, `<sys/time.h>` for accurate timestamp generation via `gettimeofday`, and `<sys/ioctl.h>` for querying terminal dimensions at runtime.

B. Custom Memory Management

Standard heap allocation (`malloc`/`free`) was replaced with a custom static bump allocator. A 1 MB continuous global array is pre-allocated at compile time inside `memory.c`. `my_alloc(size)` advances an internal offset pointer and returns the next available address, achieving $O(1)$ allocation. Deallocation is handled via `my_dealloc(ptr)`, which resets the offset back to the address of the given pointer, reclaiming all memory allocated after that point in LIFO order. At the start of each typing round, the current pool offset is captured as `sentence_start`; `my_dealloc(sentence_start)` is invoked at round completion to reclaim the sentence buffer, state arrays, and overflow buffers in a single operation. This eliminates per-object deallocation overhead while maintaining memory safety within the program's defined bounds.

C. Real-Time Terminal Interface

TermType utilizes POSIX `termios` structures to transition the user's terminal from standard canonical (cooked) mode into raw mode. This allows the application to intercept keystrokes instantaneously without requiring the user to press the `Enter` key. The user interface is driven by ANSI escape sequences, facilitating absolute cursor positioning, Unicode box drawing, and dynamic text coloring to visually indicate correct, incorrect, overflow, and untyped keystrokes in real time. Terminal width is queried on every render frame via `ioctl(STDOUT_FILENO, TIOCGWINSZ, &ws)`, allowing the typing text box to dynamically span the full width of the terminal regardless of window size. To eliminate

rendering flicker caused by multiple small `write()` syscalls, all terminal output is accumulated in a 64 KB in-process frame buffer and flushed atomically at the end of each render cycle via a single `write()` call.

D. Performance Analytics Engine

Typing performance is quantified using four primary metrics: adjusted WPM, accuracy, correct keypresses, and per-key performance.

- **Adjusted WPM**: Timestamps are recorded at the first keypress and the final character of the sentence using `gettimeofday`. Only words in which every character was typed correctly and with no overflow are counted as correct words. WPM is then computed as the number of correct words divided by elapsed time in minutes, using custom integer arithmetic to avoid floating-point dependencies. This aligns with the MonkeyType scoring standard.
- **Keystroke Accuracy**: Every physical key press is classified as correct or incorrect. Normal character presses are correct if the typed character matches the expected character, and incorrect otherwise. A backspace over a previously incorrect character is classified as a correct press; a backspace over a correct character is incorrect. Spacebar presses are correct only when the entire current word has been typed correctly with no overflow characters. Overflow presses are always classified as incorrect. Accuracy is reported as the ratio of correct keypresses to total keypresses.
- **Worst Keys Tracking**: An internal per-character map tracks the total and correct press counts for each key actually pressed during a session. At round completion, characters are sorted by ascending accuracy and the five worst-performing keys are displayed, providing targeted feedback on specific weaknesses.

E. CLI Configuration

TermType accepts an optional integer argument specifying the number of words to generate per session, with a default of 50 and a valid range of 1 to 1000. The argument is parsed manually character by character without `atoi` or any standard parsing utilities, consistent with the zero-library constraint.

III. Future Scope

The current iteration of TermType successfully demonstrates a functional zero-dependency architecture. Several avenues exist for future enhancement:

- **Dynamic Dictionary Parsing**: Transitioning from a static internal word bank to dynamic file I/O for loading external dictionaries or coding-syntax typing tests using raw `read()` syscalls.
- **Historical Progress Tracking**: Implementing raw file writing via `write()` to preserve user statistics across multiple sessions, enabling long-term progression analysis.
- **Sudden Death and Custom Modes**: Adding session modes such as sudden-death on first error or timed sessions of fixed duration, extending TermType's utility for competitive typing practice.

References

[1] Monkeytype, "A minimalistic, customizable typing test," [Online]. Available: https://monkeytype.com/. [Accessed: April 27, 2026].

[2] IEEE, "termios.h - terminal I/O interfaces," POSIX.1-2017 Standard. [Online]. Available: https://pubs.opengroup.org/onlinepubs/9699919799/basedefs/termios.h.html.

[3] TypeRacer, "The global typing competition," [Online]. Available: https://play.typeracer.com/.

[4] Free Software Foundation, "The GNU C Library Reference Manual," [Online]. Available: https://www.gnu.org/software/libc/manual/.

[5] ECMA International, "Control Functions for Coded Character Sets," ECMA-48 5th Edition, 1991.

[6] The Open Group, "ioctl - control a STREAMS device," POSIX.1-2017. [Online]. Available: https://pubs.opengroup.org/onlinepubs/9699919799/functions/ioctl.html.